



FlashAttention-T: Towards Fully Tensorized Attention by Exploiting Tensor-Vector Parallelism

Jianxing Xu*

University of Science and
Technology of China
Hefei, China
xujianxing@mail.ustc.edu.cn

Yuanbo Wen

State Key Lab of Processors
Institute of Computing
Technology, Chinese
Academy of Sciences
Beijing, China
wenyuanbo@ict.ac.cn

Jun Bi

State Key Lab of Processors
Institute of Computing
Technology, Chinese
Academy of Sciences
Beijing, China
bijun@ict.ac.cn

Ruibai Xu*

University of Science and
Technology of China
Hefei, China
ruibaixu@mail.ustc.edu.cn

Guanglin Xu

State Key Lab of Processors
Institute of Computing
Technology, Chinese
Academy of Sciences
Beijing, China
xuguanglin@ict.ac.cn

Rui Zhang

State Key Lab of Processors
Institute of Computing
Technology, Chinese
Academy of Sciences
Beijing, China
zhangrui@ict.ac.cn

Wei Li

State Key Lab of Processors
Institute of Computing
Technology, Chinese
Academy of Sciences
Beijing, China
liwei2017@ict.ac.cn

Ling Li[†]

Intelligent Software
Research Center
Institute of Software,
Chinese Academy of
Sciences
Beijing, China
liling@iscas.ac.cn

Tianshi Chen

Cambricon Technologies
Beijing, China
tchen@cambricon.com

Qi Guo[‡]

State Key Lab of Processors
Institute of Computing
Technology, Chinese
Academy of Sciences
Beijing, China
guoqi@ict.ac.cn

Yunji Chen[†]

State Key Lab of Processors
Institute of Computing
Technology, Chinese
Academy of Sciences
Beijing, China
cyj@ict.ac.cn

Abstract

The attention mechanism is central to modern deep learning, particularly in large language models (LLMs), but suffers from quadratic computational complexity. To accelerate attention computation on GPUs, fused attention techniques (e.g., FlashAttention) consolidate the matrix multiplication (GEMM) and softmax computations into a single kernel. However, these operations remain computationally decoupled: the GEMM leverages high-performance tensor units (Tensor Cores), while the softmax executes on slower vector units (CUDA cores). This imbalance induces severe *vector intervals*—periods where tensor units sit idle awaiting vector unit completion—significantly underutilizing tensor units.

*Also with State Key Lab of Processors, Institute of Computing Technology, Chinese Academy of Sciences.

[†]Also with University of Chinese Academy of Sciences.

[‡]Qi Guo is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.

PPoPP '26, Sydney, NSW, Australia

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2310-0/2026/01

<https://doi.org/10.1145/3774934.3786425>

Furthermore, ongoing hardware advancements delivering faster tensor units exacerbate this bottleneck.

To resolve the vector interval bottleneck, in this paper, we propose FlashAttention-T, a fused attention implementation that advances toward fully tensorized fused attention. Our key insight is to offload critical softmax primitives to idle tensor units, maximizing hardware utilization and throughput. Concretely, we first introduce a series of operand value assignment methods to *repurpose tensor matrix multiply-add (MMA) instructions* for executing softmax primitives (e.g., element-wise scaling). Building on this, we design a *tensorized online softmax algorithm* with numerical stability guarantees, adhering to the constraints of repurposed tensor MMA instructions. To maximize performance, we *parallelize the online softmax computation across tensor and vector units* via architecture-aware scheduling techniques, fully leveraging heterogeneous parallelism.

Extensive evaluation of various attention configurations across multiple platforms including NVIDIA Ampere (A100 and AGX Orin) and Hopper (H100) GPUs demonstrates that FlashAttention-T achieves vector interval ratios 1.17–2.18× lower than the baseline on Ampere GPUs and reduces them down to 2.7% on the Hopper H100, with average speedups of

up to 1.17 $\times$ against FlashAttention-2 and FlashAttention-3 while preserving numerical stability and accuracy.

CCS Concepts: • Computing methodologies → Artificial intelligence; Parallel computing methodologies.

Keywords: Fused Attention, GPU Kernel, Tensor Core, Tensor-Vector Parallelism

ACM Reference Format:

Jianxing Xu, Yuanbo Wen, Jun Bi, Ruibai Xu, Guanglin Xu, Rui Zhang, Wei Li, Ling Li, Tianshi Chen, Qi Guo, and Yunji Chen. 2026. FlashAttention-T: Towards Fully Tensorized Attention by Exploiting Tensor-Vector Parallelism. In *Proceedings of the 31st ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPOPP '26)*, January 31 – February 4, 2026, Sydney, NSW, Australia. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3774934.3786425>

1 Introduction

The attention mechanism, which computes complex contextual relationships across input sequences, is fundamental to modern deep learning, particularly in large language models (LLMs). However, it incurs significant computational cost with quadratic scaling with respect to the sequence length [31]. As the sequence length continues to increase for real-life applications [3, 32, 33], the attention mechanism constitutes the primary computational constraint of LLMs.

To accelerate attention computation on commodity hardware like GPUs, fused attention implementations, exemplified by FlashAttention [6], have emerged by consolidating the key matrix multiplication (GEMM) and softmax operations into a single kernel, significantly reducing memory access and improving throughput. Subsequent works, FlashAttention-2 [5] and FlashAttention-3 [26], further enhance this technique by optimizing work partitioning, improving parallelism, and leveraging hardware-specific features. As a result, the FlashAttention series implementations have been widely deployed in various LLM inference frameworks including PyTorch [24], vLLM [14], and SGLang [43].

However, current fused attention implementations such as FlashAttention-2 and FlashAttention-3 are severely constrained by the *vector intervals*—periods where the high-performance tensor units (Tensor Cores) stay idle while awaiting slower vector units (CUDA cores) to complete, causing underutilization of the tensor units. As shown in Figure 1(a), the vector interval consumes up to 36% of the iteration cycles with an attention head dimension size of 128 used by various well-known LLMs (e.g., Llama2, Llama3, and Qwen3 [8, 30, 36]), and the fraction increases to up to 42% at head dimension size of 64 used by the pioneering model gpt-oss [23]. This inefficiency stems from computational decoupling of the key GEMM and softmax operations: the GEMM leverages tensor units whereas the softmax executes on vector units. Such performance heterogeneity between these units induces significant tensor unit idleness. Moreover,

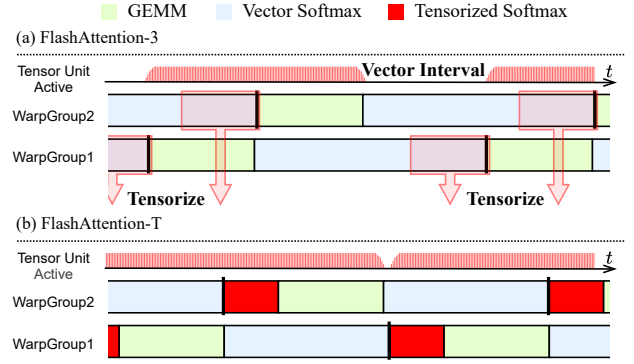


Figure 1. The execution timelines of (a) FlashAttention-3 and (b) FlashAttention-T on Hopper GPUs. FlashAttention-T resolves the vector interval bottleneck by tensorizing key softmax primitives and exploiting thread-level parallelism between the vector and tensorized softmax computations.

ongoing hardware advancements, such as faster tensor units and lower-precision GEMM support [13, 15], exacerbate this performance bottleneck.

Tensorizing the softmax computation offers a promising solution to utilize the idle tensor units and address this bottleneck, but it poses significant challenges across three dimensions. At the *instruction level*, the tensor matrix multiply-add (MMA) instructions, originally designed for GEMM operations, must be repurposed to implement critical softmax primitives (i.e., element-wise scaling, fused multiply-add, and row-sum reduction). This requires careful matrix operand preparation and result interpretation under intricate cross-thread fragment layouts, while minimizing overhead. At the *algorithm level*, the repurposed tensor MMA instructions impose algorithmic constraints, complicating the design of a tensorized softmax algorithm with numerical stability guarantees. At the *architecture level*, the complexity and diversity of modern hardware necessitate architecture-aware scheduling techniques that map the tensorized softmax algorithm to the tensor and vector units in parallel, exploiting tensor-vector parallelism for optimal performance.

In this paper, we present FlashAttention-T, a fused attention implementation that advances toward fully tensorized fused attention. FlashAttention-T repurposes tensor MMA instructions to tensorize critical softmax primitives, introduces a tensorized online softmax algorithm with numerical stability guarantees, and implements architecture-aware scheduling to exploit tensor-vector parallelism in the softmax computation. Experimental results show that FlashAttention-T achieves vector interval ratio reductions of 1.17–2.18 $\times$ on Ampere GPUs and down to 2.7% on Hopper H100, with average speedups of up to 1.17 $\times$ over FlashAttention-2 and FlashAttention-3, while preserving numerical stability and accuracy. Figure 1(b) shows how we tensorize softmax computation and exploit thread-level tensor-vector parallelism to resolve the vector interval bottleneck on Hopper GPUs.

To the best of our knowledge, this work is *the first trial to fully tensorize the execution of attention operation*. The main contributions are summarized as follows:

- **Tensor MMA-based softmax primitives.** We propose a series of operand value assignment methods that repurpose GPU tensor MMA instructions to implement critical softmax primitives, enabling these operations to execute on the tensor unit with minimal overhead.
- **Tensorized online softmax algorithm.** We propose a tensorized online softmax algorithm that enables tensorizing all compatible softmax computations within fused attention, while providing numerical stability guarantees.
- **Tensor-vector scheduling techniques.** We introduce architecture-aware scheduling techniques that parallelize the online softmax computation across the tensor and vector units, exploiting instruction-level and thread-level tensor-vector parallelism to minimize the vector interval while maximizing attention throughput.
- **Extensive evaluation.** We evaluate FlashAttention-T's performance and accuracy on multiple GPU platforms across diverse attention configurations.

2 Motivation

In this section, we introduce the key insights motivating our work to tensorize softmax computation within fused attention. We begin by examining the algorithmic flow of fused attention, demonstrating that softmax computation constitutes a significant time fraction of the attention computation across sequence lengths. We then analyze the program execution timelines of two state-of-the-art implementations, FlashAttention-2 (optimized for NVIDIA Ampere GPUs) and FlashAttention-3 (optimized for Hopper GPUs). With cycle-level profiling, our analysis reveals two critical insights: (1) The vector-based softmax in current implementations causes substantial *vector intervals*, during which the tensor unit remains idle awaiting vector unit completion. (2) Advances in hardware, such as lower-precision GEMM (e.g., FP8) and faster tensor units, exacerbate this inefficiency, underscoring the need for a tensorized softmax tailored to evolving GPU architectures.

2.1 The Algorithmic Flow of Fused Attention

As depicted in Figure 2(b), fused attention parallelizes the computation across warpgroups (or threadblocks) in the GPU kernel, with each warpgroup computing a row block of the attention output matrix O . For each warpgroup, the attention row block is further tiled using safe online softmax [5, 6, 16] to compute and accumulate the attention output row block O in s/b_N iterations, where s is the sequence length and b_N is the tile size along the sequence dimension.

Figure 2(a) illustrates the algorithmic flow of each attention output accumulation iteration, which involves the following stages: ① compute an attention logit tile S , ② update

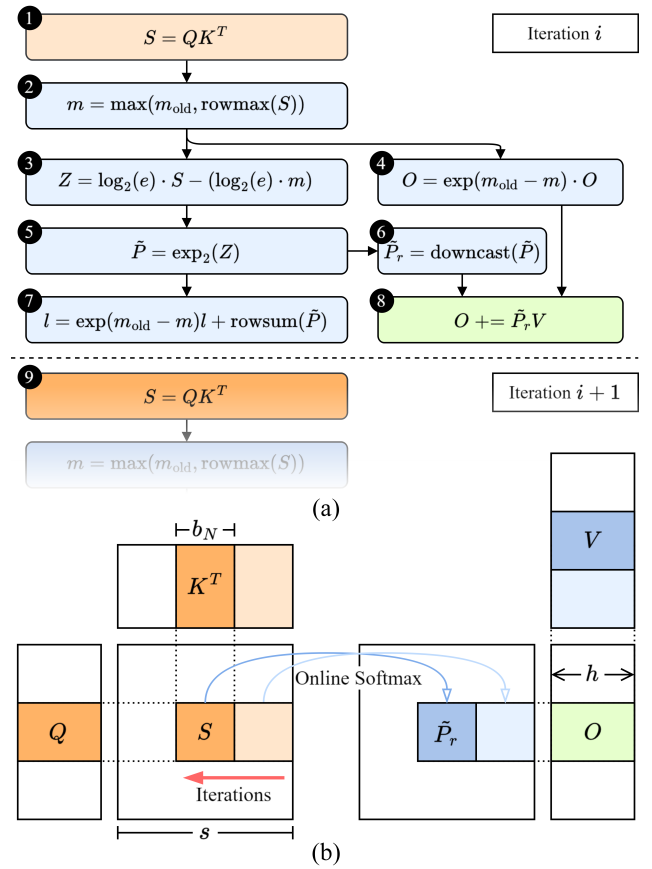


Figure 2. (a) The algorithmic flow of fused attention iterations within a warpgroup. (b) The algorithmic flow iterations visualized in the attention computation space.

the row maximum values m of the attention logit row block, ③⑤⑥ exponentiate the S tile to produce the attention score tile $\tilde{P}_r$ based on the updated m and previous row maximum m_{old} , ④ rescale the attention output O , ⑦ update the attention score row sum l , and ⑧ accumulate the $\tilde{P}_r V$ results onto O . A key strength of this tiled scheme is that it ensures all intermediate softmax results reside in registers. This minimizes memory accesses and enables tensorization of softmax primitives using our repurposed tensor MMA instructions that operate directly on register data.

To quantify the aggregated online softmax computational cost over the s/b_N iterations for the whole attention computation, we analyze the operations in the process: ② max (maximum), ③ fma (fused multiply-add, FMA), ④ mul (multiply), ⑦ add (addition), and ⑤ exp (exponential). For sequence length s and head dimension size h , the primitive operation counts are: max: s^2 , fma+add+mul: $2s^2 + sb_N^{-1} \cdot sh$, and exp: s^2 , totaling $4s^2 + b_N^{-1}s^2h$, compared to $2s^2h$ FMAs for GEMM operations. Under the current paradigm, where softmax operations use vector units (with effective throughput k) and GEMM operations use tensor units (with throughput c), the softmax time fraction in this computation, defined as

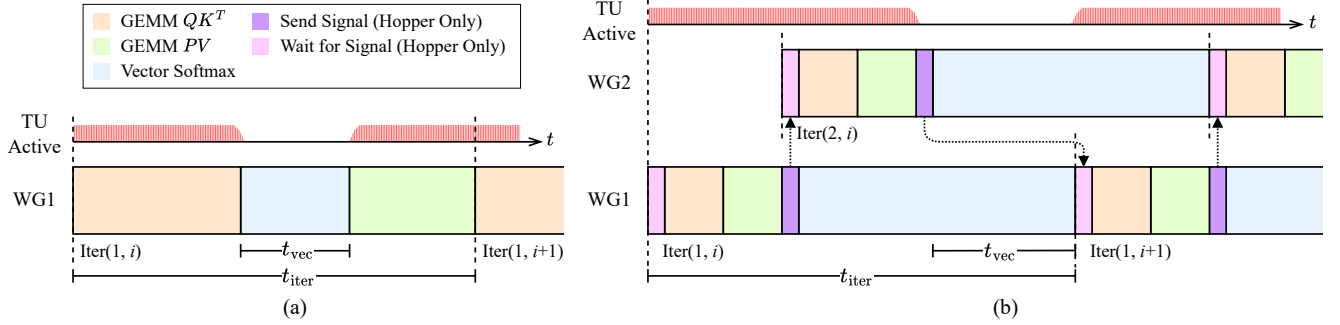


Figure 3. The execution timelines of (a) FlashAttention-2 on Ampere GPUs, and (b) FlashAttention-3 on Hopper GPUs. Both implementations suffer from the vector interval bottleneck due to their exclusive reliance on the vector unit for softmax computation.

$T_{\text{softmax}} / (T_{\text{softmax}} + T_{\text{GEMM}})$, where T_{softmax} and T_{GEMM} denote the execution times of softmax and GEMM operations, is:

$$\frac{T_{\text{softmax}}}{T_{\text{softmax}} + T_{\text{GEMM}}} = \frac{c(4 + b_N^{-1}h)}{c(4 + b_N^{-1}h) + 2kh}$$

It can be observed that given a constant head dimension size h and a high tensor-vector throughput ratio c/k , the softmax runtime takes a significant fraction of the computation, independent of sequence length s . Moreover, hardware advancements, such as the NVIDIA Hopper H100 GPU's doubled FP16-FP32 GEMM throughput over the Ampere A100 [19, 20] and further doubling with FP8-FP32 precision [20], amplify this fraction as c/k increases. This underscores an urgent need for optimizing softmax computation within fused attention.

2.2 Revealing the Vector Interval Bottleneck

To reveal the vector interval bottleneck induced by vector-based softmax computation, we analyze the execution timelines of state-of-the-art fused attention implementations on their optimized GPU architectures: (a) FlashAttention-2 on Ampere GPUs and (b) FlashAttention-3 on Hopper GPUs.

As depicted in Figure 3(a), FlashAttention-2 employs sequential scheduling. In each iteration, the warpgroup (WG) first executes synchronous tensor MMA instructions for the QK^T computation on the tensor unit (TU), followed by various vector instructions for softmax computation on the vector unit, and finally another series of tensor MMA instructions for the PV computation. This sequential scheduling creates a vector interval spanning t_{vec} cycles, where the tensor unit remains idle awaiting the vector unit's completion of softmax computation, delaying the PV computation.

FlashAttention-3 leverages the Hopper GPU's asynchronous warpgroup MMA (WGMMA) feature to enable pipelined scheduling, as shown in Figure 3(b). Let $\text{Iter}(w, i)$ denote the i -th iteration of warpgroup w of the program. In a typical two-warpgroup setting, $\text{Iter}(1, i)$ begins by committing a batch of asynchronous WGMMA instructions for QK^T , followed by another batch for the PV computation. This is enabled by shifting the algorithm's iteration boundary from

below 78 to below 76 in Figure 2. After the PV WGMMA completion, warpgroup 1 sends signal to warpgroup 2. Upon receiving the signal, warpgroup 2 starts executing its WGMMA instructions for $\text{Iter}(2, i)$ in parallel with the vector instructions for $\text{Iter}(1, i)$. This pipelined scheduling enables the warpgroups to partially overlap their GEMM and softmax computations. However, the non-overlapped softmax computation in each iteration still creates a vector interval of t_{vec} cycles, during which the tensor unit remains idle.

To quantify the vector interval time t_{vec} , the iteration time t_{iter} , and the *vector interval ratio* $t_{\text{vec}}/t_{\text{iter}}$, we conduct cycle-level profiling on FlashAttention-2 and FlashAttention-3 using cycle-counting routines under a typical attention configuration ($h = 128$, $s = 4096$). As shown in Table 1, the vector interval occupies an average of 29.8% of the iteration cycles in FlashAttention-2, and rises to 36.3% in FlashAttention-3 with FP8-FP32 GEMM. These results confirm the vector interval bottleneck and its exacerbation on modern hardware, motivating our work to tensorize softmax computation for improved tensor unit utilization and attention throughput.

Table 1. Vector interval profiling results for FlashAttention.

Implementation	GPU	Precision	t_{vec}	t_{iter}	$t_{\text{vec}}/t_{\text{iter}}$
FlashAttention-2	A100	FP16-FP32	924	3100	29.8%
FlashAttention-3	H100	FP8-FP32	1126	3106	36.3%

3 Repurposing Tensor MMA Instructions

To provide a robust foundation for tensorizing the softmax computation in fused attention kernels, we propose novel operand value assignment methods that repurpose tensor MMA instructions for key primitives in the softmax exponentiation and normalization steps. These methods efficiently implement element-wise scaling, fused multiply-add, and row-sum reduction, minimizing permutation-induced overhead caused by repurposing.

Tensor MMA instructions in modern GPUs partition their logical operand matrices into *fragments* distributed across threads, where a fragment is a vector of matrix elements stored in contiguous registers. In this section, we use $X[i, j]$

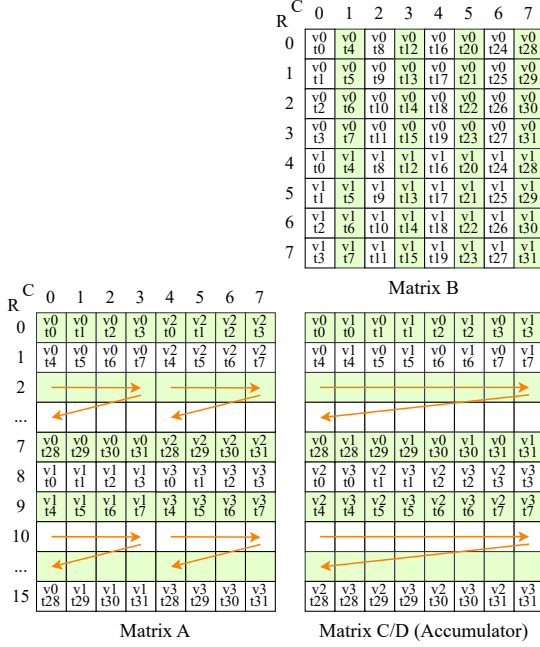


Figure 4. Logical matrix element to fragment mapping of tensor MMA instruction HMMA.1688.F32.TF32 found in modern NVIDIA GPUs. Colors in this figure are for illustrative purpose only.

to denote the element at row i and column j of the logical matrix X , and $X(v, t)$ to denote the v -th element in the fragment held by thread t , where $v \in \{0, 1, \dots, |X| - 1\}$ and $t \in \{0, 1, \dots, T - 1\}$, with T being the number of participating threads and $|X|$ the number of elements per fragment. The mapping function $\mu_X : X[i, j] \rightarrow X(v, t)$ is architecture-specific and varies by operand type (e.g., A, B, C in $D = AB + C$). An example is shown in Figure 4.

3.1 Element-Wise Scaling and Fused Multiply-Add

We begin by describing the method for repurposing the tensor MMA instruction to implement element-wise scaling with the C fragment set to zero. Given the scaling factor α , our goal is to determine a value assignment to the fragment B such that the tensor MMA instruction computes an output fragment D that yields a *permuted* element-wise scaling of the input fragment A :

$$D(v, t) = \alpha A(\sigma(v), t)$$

where $\sigma \in S_{|A|}$ is a permutation in the symmetric group on $\{0, 1, \dots, |A| - 1\}$, and $|A| = |D|$ is the number of elements per fragment. This inevitable permutation arises due to the tensor unit's internal fragment mapping, which may reorder elements within each thread's registers. For example, with $|A| = 4$, a valid value assignment to fragment B under a permutation $\sigma = (1\ 2)$ (in cycle notation) yields:

$$\begin{aligned} D(0, t) &= \alpha A(0, t), & D(1, t) &= \alpha A(2, t), \\ D(2, t) &= \alpha A(1, t), & D(3, t) &= \alpha A(3, t). \end{aligned}$$

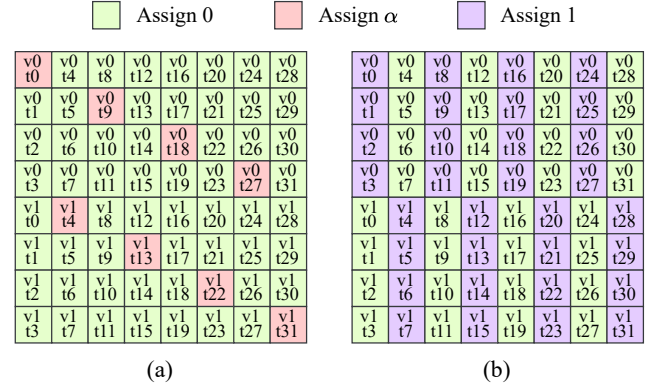


Figure 5. Optimal fragment B value assignment methods for repurposing tensor MMA instruction HMMA.1688.F32.TF32 to implement (a) element-wise scaling by factor α or fused multiply-add, and (b) row-sum reduction.

The form of this permutation is determined by the value assignment to fragment B , and it introduces *swap overhead* as reordering $D(v, t)$ to achieve the non-permuted output (i.e., $D(v, t) = \alpha A(v, t)$) requires swap operations within each thread's registers. Each swap increases instruction count and register pressure, causing swap overhead. To quantify this overhead, we use the *Cayley distance* [25] of σ , defined as the minimum number of arbitrary swaps needed to transform σ into the identity permutation. To achieve a functional value assignment while minimizing this overhead, we formulate the problem as a constrained optimization problem: given $\lambda(v, t) \in \{0, \alpha\}$ as the value assigned to $B(v, t)$, minimize the Cayley distance $d_C(\sigma)$ subject to:

$$\begin{aligned} D(v, t) &= \mu_D(D[i, j]) \\ &= \sum_k \mu_A(A[i, k]) \mu_B(B[k, j]) = \alpha A(\sigma(v), t) \end{aligned}$$

This problem can be solved empirically or with solvers. Figure 5(a) illustrates an optimal fragment B value assignment for $\sigma = (1\ 2)$, achieving a Cayley distance $d_C(\sigma) = 1$ with only one swap to recover the non-permuted output. Building on this, the same value assignment to fragment B extends to a fused multiply-add repurposing with a constant scaling factor α and an arbitrary offset $C(v, t)$:

$$D(v, t) = \alpha A(\sigma(v), t) + C(v, t)$$

This extension is achieved by adding $C(v, t)$ to the MMA output leveraging the tensor unit's accumulator, preserving the permutation σ induced by B .

3.2 Row-Sum Reduction

Beyond scaling and fused multiply-add, we extend our approach to repurpose tensor MMA instructions for row-sum reduction. As shown in Figure 2(a), this operation $\textcircled{7}$ involves summing the exponents over rows of the attention score matrix $\tilde{P}$, which shares the fragment layout of matrix D as in Figure 4. Our goal is to compute a fragment $s(v, t)$ with two

elements per thread, satisfying:

$$s(0, t) = \sum_{v \in \{0,1\}} \sum_{t \in \kappa_i} D(v, t), \quad (1)$$

$$s(1, t) = \sum_{v \in \{2,3\}} \sum_{t \in \kappa_i} D(v, t). \quad (2)$$

where κ_i denotes quad-pairs of threads in the same row of the matrix D (e.g., threads 0–3, 4–7).

Current implementations perform this summation in two steps: (1) intra-thread additions, computing $D(0, t) + D(1, t)$ and $D(2, t) + D(3, t)$, and (2) inter-thread all-reduce operations within each quad-pair κ_i . This approach incurs overhead due to explicit thread synchronization and communication. We propose using the tensor MMA instruction to perform the summation more efficiently. By setting the input fragment $A(v, t) = D(v, t)$ (from matrix $\tilde{P}$), assigning fragment B as shown in Figure 5(b), and setting $C(v, t) = 0$, the MMA instruction computes an output fragment $D'(v, t)$:

$$D'(v, t) = \sum_{t \in \kappa_i} A(\sigma(v), t) = \sum_{t \in \kappa_i} D(\sigma(v), t)$$

where $v \in \{0, 1, \dots, |A| - 1\}$, $|A| = |D'| = 4$, and $\sigma = (1\ 2)$. To obtain $s(v, t)$, we perform two intra-thread additions:

$$s(0, t) = D'(0, t) + D'(2, t), \quad (3)$$

$$s(1, t) = D'(1, t) + D'(3, t). \quad (4)$$

These additions yield a result matching Eqs. (1) and (2), with the permutation σ absorbed into the intra-thread additions, incurring no additional swap overhead.

3.3 Discussion

Our methods for repurposing tensor MMA instructions establish a robust foundation for tensorizing the softmax computation in fused attention kernels. We discuss three critical aspects of the repurposing: (1) **Algorithmic constraints**. The scaling and fused multiply-add repurposing require the scaling factor α to be uniform across all rows of the input matrix A for the tensor MMA instruction. This necessitates a tensorized softmax algorithm to enable the use of the repurposed instructions, as detailed in Section 4. (2) **Architectural availability**. Our approach requires $|A| = |D|$, supporting synchronous MMA instructions like HMMA.1688.F32.TF32 available in all modern NVIDIA GPU architectures (e.g., Ampere and Hopper). These methods also extend to the Hopper-specific asynchronous WGMMA instructions such as HGMMA.64x8x8.F32.TF32, with the caveat that fragment B must be in shared memory [21]. (3) **No copy overhead**. One key advantage of the repurposed tensor MMA instructions is that they operate directly on the GEMM tensor MMA instruction's output fragments, in exactly the same memory space (i.e., the registers on both Ampere and Hopper GPUs). Thus, tensorizing the softmax primitives using our repurposing incurs no extra copy overhead.

4 Tensorized Online Softmax Algorithm

The repurposed tensor MMA instructions for scaling, fused multiply-add (FMA), and row-sum reduction build a robust foundation for tensorizing the ④ attention output rescaling, ③ attention logits rescaling, and ⑦ attention score row summation as illustrated in Figure 2, respectively. However, the original fused attention algorithm's ④ output rescaling computation, $O = \exp(m_{\text{old}} - m) \cdot O$, requires a per-row scaling factor $\exp(m_{\text{old}} - m)$. This conflicts with the uniform scaling factor α constraint of the repurposed tensor MMA scaling instructions. To resolve this conflict, we propose a tensorized online softmax algorithm that ensures numerical stability with high probability. The core of this algorithm is a novel surrogate maximum.

The fused attention algorithm uses safe online softmax to prevent numerical overflow during exponentiation of the attention logit matrix S . As shown in Figure 2, this method maintains a running row maximum $m[i]$ for the i -th row of S , subtracting $m[i]$ from each element in that row before exponentiation to produce the attention score matrix $\tilde{P}$. Formally, let $S[i, j]$ denote the element at the i -th row and j -th column of S , and $m[i]$ the maximum value of the i -th row. This ensures that $\forall i, \forall j, 0 \leq \exp(S[i, j] - m[i]) \leq 1$, keeping all exponentiated values within a representable range.

Our key insight is that the softmax remains numerically stable by replacing the row maximum $m[i]$ with a surrogate $\hat{m}[i]$, provided two failure conditions are avoided:

- **Any-overflow**. For some row i , there exists a column j such that $\exp(S[i, j] - \hat{m}[i]) > F_{\text{max}}$, where F_{max} is the maximum representable floating-point value, resulting in an indeterminate infinity-over-infinity form.
- **All-underflow**. For some row i , all $\exp(S[i, j] - \hat{m}[i])$ are effectively zero, leading to a zero-over-zero failure.

To address the uniform scaling factor constraint of the tensor MMA scaling instructions, we propose a surrogate maximum $\hat{m}[i]$ defined as the maximum value across an X -row tile containing row i of the attention logit matrix S :

$$\hat{m}[i] = \max(\{S[i', j'] : j' \in [0, s), i' \in [X \lfloor \frac{i}{X} \rfloor, X \lfloor \frac{i}{X} \rfloor + X)\})$$

where s is the sequence length. This surrogate generalizes to the repurposed HMMA.1688 ($X=16$) and HGMMA.64x8x8 ($X=64$) scaling instructions and future tensor MMA instructions. Unlike FlashDecoding++'s static maximum [11], our X -row tile-based $\hat{m}[i]$ dynamically adapts to local row distributions, ensuring stability for diverse attention patterns with high probability by offering two properties:

- **No overflow** Since $\hat{m}[i] \geq m[i]$, this surrogate maximum ensures $\exp(S[i, j] - \hat{m}[i]) \leq \exp(S[i, j] - m[i]) \leq 1$, strictly preventing indeterminate forms.
- **Negligible all-underflow probability** Although a larger $\hat{m}[i]$ increases the chance of individual exponents underflowing, the joint probability of *all* exponents in a row

underflowing is asymptotically small for typical attention score distributions[9, 11, 12] (e.g., Gaussian).

For extreme attention score distributions triggering all-underflow failure with the proposed surrogate, the implementation can selectively revert attention output rescaling to vectorized computation, bypassing the surrogate $\hat{m}[i]$ while keeping all other compatible primitives tensorized¹. Nevertheless, experimental results in Section 7 confirm that the surrogate $\hat{m}[i]$ causes no numerical failures and maintains attention accuracy close to the baseline.

Algorithm 1 Tensorized Online Softmax for Fused Attention

Require: Attention logit matrix $S \in \mathbb{R}^{n \times s}$, attention output matrix $O \in \mathbb{R}^{n \times d}$, previous attention logit row maximums $m_{\text{old}} \in \mathbb{R}^n$, running attention score row sums $l \in \mathbb{R}^n$, and surrogate tile size X .

Ensure: Attention score matrix $\tilde{P} \in \mathbb{R}^{n \times s}$, updated O , m , and l .

- 1: Compute X -row tile maxima: $\hat{m} \leftarrow \text{tilemax}(S, X) \quad \triangleright \hat{m} \in \mathbb{R}^{\lceil \frac{n}{X} \rceil}$
 - 2: Get old surrogate maximums: $\hat{m}_{\text{old}} \leftarrow m_{\text{old}}[X \cdot i]$ for $i \in [0, \lceil n/X \rceil)$
 - 3: Rescale attention output with repurposed tensor scaling instructions:
 $O \leftarrow \exp(\hat{m}_{\text{old}} - \hat{m}) \cdot O \quad \triangleright X$ -row uniform $\hat{m}$ enables tensorization
 - 4: Assign surrogate maximums: $m[i] \leftarrow \hat{m}[\lfloor i/X \rfloor]$ for $i \in [0, n)$
 - 5: Rescale attention logits with repurposed tensor FMA instructions:
 $Z \leftarrow \log_2(e) \cdot S - (\log_2(e) \cdot m)$
 - 6: Produce attention score matrix with vector exponentiation:
 $\tilde{P} \leftarrow \exp_2(Z)$
 - 7: Update attention score row sums with repurposed tensor row-sum reduction instructions:
 $l \leftarrow \exp(m_{\text{old}} - m)l + \text{rowsum}(\tilde{P})$
 - 8: **return** $\tilde{P}, O, m, l$
-

With the proposed surrogate $\hat{m}$, we present the tensorized online softmax algorithm for a warpgroup within the fused attention iterations in Algorithm 1. Compared to the original algorithm in Figure 2, it substitutes the true row maximum m with $\hat{m}$, and tensorizes all compatible softmax primitives.

5 Scheduling for Tensor-Vector Parallelism

The repurposed tensor MMA instructions and the tensorized softmax algorithm enable tensorizing all compatible softmax computations in fused attention. However, our preliminary results indicate that an all-tensorized scheme, where all compatible softmax computations are tensorized, yields only marginal performance gains. The main reason is that the raw throughput of tensor MMA instructions in current hardware has not reached a turning point. For instance, the HMMMA.1688.F32.TF32 instruction on the A100 has a reciprocal throughput of 8 [1], yielding an effective throughput of 16 elements per cycle across all repurposing methods, which exactly matches the vector unit’s throughput. This indicates that such all-tensorized scheme is a sub-optimal strategy.

To address this challenge, we develop architecture-aware scheduling techniques that parallelize the softmax computation across tensor and vector units, exploiting instruction-level parallelism (ILP) and thread-level parallelism (TLP)

¹Note that the attention logits rescaling operation only requires a constant scaling factor $\log_2(e)$.

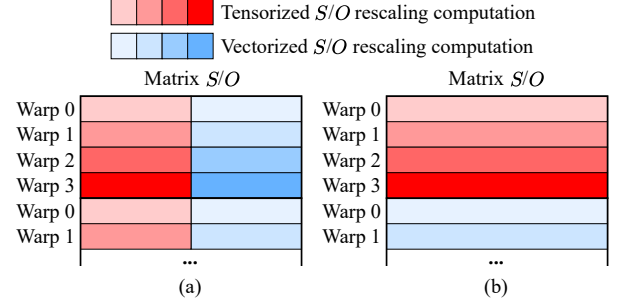


Figure 6. FlashAttention-T horizontal and vertical split strategies for ILP scheduling on Ampere GPUs. In (a) the computation on the matrix tiles of a warp are horizontally split into tensorized and vectorized portions, while in (b) the matrix tiles are vertically split.

between the units to maximize tensor unit utilization and attention throughput. Concretely, we propose ILP and TLP scheduling techniques tailored to Ampere and Hopper GPUs, respectively. The common idea of the two strategies is to first split the softmax computation into tensorized and vectorized portions, and then parallelize the two portions using specific techniques suitable for each GPU architecture.

5.1 ILP Scheduling for Ampere GPUs

On Ampere GPUs, FlashAttention-T splits the softmax computation into tensorized and vectorized portions, and schedules ILP interleaving of the tensor and vector softmax instructions within each warp.

Tensor-vector split strategies. In the baseline implementation FlashAttention-2, the S and O attention row block handled by each warpgroup (or threadblock) is further tiled into 16-row tiles, with each 16-row tile cyclically assigned to each warp in the warpgroup. Based on this tiled scheme, FlashAttention-T first implements the 16-row maximum surrogate $\hat{m}$ using warp all-reduce instruction REDUX to enable tensorizing the attention logits S rescaling, attention output O rescaling, and attention score $\tilde{P}$ row-summation with the repurposed tensor MMA instructions. It then selects the most performant method from the following strategies to split the S and O rescaling operations into tensorized and vectorized portions, based on the attention configuration:

- **Horizontal split.** As depicted in Figure 6(a), this strategy splits the tensor and vector portions horizontally across the matrix tiles of a warp, with a variable split ratio. Our preliminary experiment results suggest a split ratio around 1:1 yields near-optimal performance.
- **Vertical split.** As shown in Figure 6(b), this strategy splits across multiple matrix tiles assigned to the same warp. Since a warp computes at most two tiles due to register capacity limit in practice, the split ratio is fixed at 1:1.

Instruction-level parallelism scheduling. To parallelize the tensorized and vectorized softmax computation, FlashAttention-T evenly interleaves the repurposed tensor instructions and vector instructions executed within each

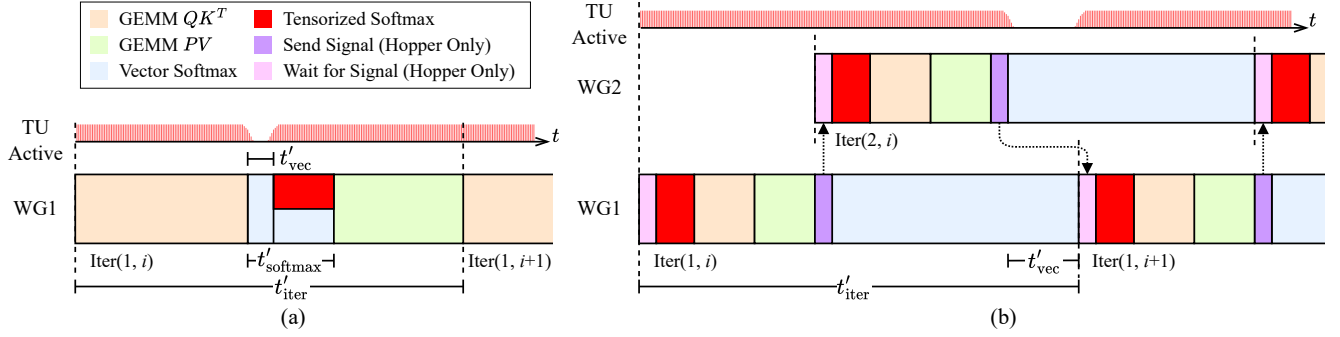


Figure 7. The execution timelines of FlashAttention-T on (a) Ampere and (b) Hopper GPUs. FlashAttention-T parallelizes the tensorized and vectorized softmax computations via ILP within warps on Ampere and via TLP across warpgroups on Hopper.

warp, such that the vector instructions run in parallel within the issue bubbles of the tensor instructions. In addition, the common B fragments for all repurposed tensor instructions of the same type are generated in advance and reused to reduce generation overhead and register bank conflicts [35]. We keep all other non-tensorizable softmax primitives such as max and exp vectorized. As shown in Figure 7(a), this scheduling reduces the softmax time from t_{vec} in Figure 3(a) to $t'_{softmax}$, in which the vector interval time is shrunk to t'_{vec} . By exploiting ILP, FlashAttention-T reduces the iteration time and also significantly improves tensor unit utilization.

5.2 TLP Scheduling for Hopper GPUs

Hopper GPUs introduce asynchronous WGMMA instructions, which enable non-blocking GEMM execution and support inter-warpgroup parallelism with vector instructions. Leveraging this, on Hopper GPUs, we aim to split the softmax computation at an algorithmic-stage granularity and parallelize the tensorized and vectorized softmax computation across warpgroups via TLP.

Tensor-vector split strategy. There are three algorithmic stages eligible for tensorization in the softmax computation: (1) attention logits S rescaling, (2) attention output O rescaling, and (3) attention score $\tilde{P}$ row-summation. Ideally, each of these stages can be either fully tensorized or fully vectorized, creating a small search space over the algorithm stages for best performance. Unfortunately, currently, NVIDIA’s nvcc compiler imposes a restrictive obstacle to this idea. Asynchronous execution of batched WGMMA instructions requires *all* accumulator operand registers to be free of any dependent access; otherwise, the compiler would serialize these instructions by placing wait barriers after each of the instructions, making them effectively synchronous and severely degrading performance. Specifically in our case, we found that tensorizing either the O rescaling or S rescaling would cause WGMMA serialization due to register access originating in dependent algorithm stages intruding into the tensorized stages, while tensorizing the $\tilde{P}$ row-summation would not, due to its leaf-stage nature minimizing cross-stage register dependencies.

As a result, we only tensorize the $\tilde{P}$ row-summation on Hopper GPUs. While this choice limits potential gains from further tensorization, it is strategic for two main reasons: (1) The row-summation contributes a significant portion of the tensorizable softmax operations (as detailed in Section 2.1). Thus, tensorizing this algorithmic stage creates a balanced tensor-vector softmax workload. (2) As discussed in Section 3.2, tensorizing the row-sum reduction primitives eliminates the need for explicit thread synchronization and communication, benefiting performance. In addition, this strategy minimizes register pressure, as the register-based accumulator fragment is reused across all WGMMA row-sum reduction instructions.

Thread-level parallelism scheduling. As shown in Figure 7(b), FlashAttention-T tensorizes the $\tilde{P}$ row-summation, and schedules the repurposed WGMMA instructions to join the QK^T and PV WGMMA batch of the warpgroup’s next iteration. This scheduling enables the tensorized computation to run in parallel with the vectorized S and O rescaling operations from other warpgroups in a TLP manner, reducing the vector interval time to t'_{vec} and improving tensor unit utilization. Note that the surrogate $\hat{m}$ is not used, as it’s only required by the matrix O rescaling. Thus, FlashAttention-T’s numerical stability properties remain the same as the baseline implementation on Hopper GPUs.

6 Evaluation Methodology

6.1 Evaluation Platforms

We extensively evaluate FlashAttention-T across diverse NVIDIA GPU platforms, encompassing both server-class and edge-class variants, and spanning various architectures. Specifically, these include: (1) NVIDIA A100 80GB SXM4, a widely deployed server-class GPU utilizing the Ampere architecture (SM80); (2) NVIDIA Jetson AGX Orin 64GB, an edge-class system designed for autonomous applications, integrating a powerful Ampere GPU (SM87); and (3) NVIDIA H100 80GB PCIe, a high-performance server-class GPU featuring the Hopper architecture (SM90A). As discussed in Section 5, FlashAttention-T is architecture-optimized, using the

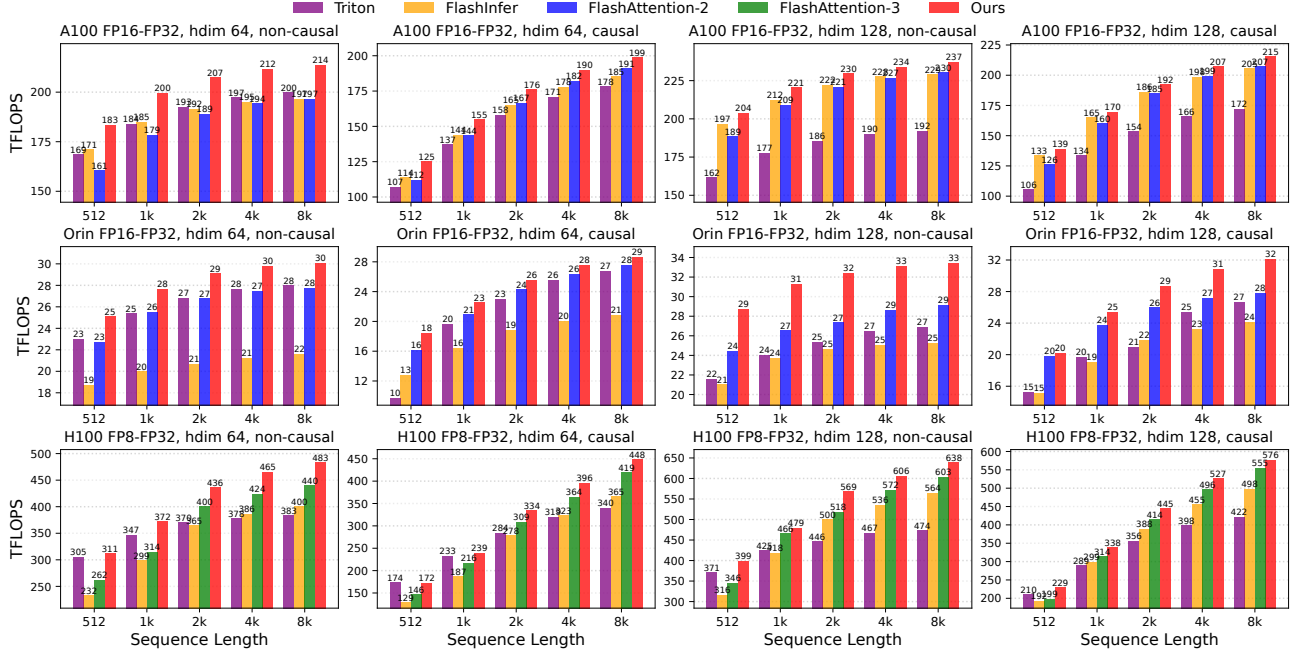


Figure 8. Attention throughput of FlashAttention-T and baselines across different configurations on Ampere and Hopper GPUs.

ILP scheduling for the Ampere GPUs (the A100 and Orin) and the TLP scheduling for the Hopper GPU (the H100).

6.2 Evaluation Design

To fully evaluate the performance of FlashAttention-T and address any potential numerical stability and accuracy concerns, we conduct five evaluations in two categories.

Performance. This category of evaluations focuses on quantifying the performance gains of FlashAttention-T. We begin with a comprehensive evaluation on the multi-head attention performance of FlashAttention-T across two data precisions, FP16-FP32 and FP8-FP32, with diverse configurations, measuring the attention throughput to assess whether FlashAttention-T matches or exceeds baseline performances.

We then conduct a cycle-level evaluation to demonstrate that FlashAttention-T enhances attention throughput by reducing the tensor unit idle time. Specifically, we calculate the vector interval ratio $t'_{\text{vec}}/t'_{\text{iter}}$ for various attention configurations and compare the results against baseline values.

Finally, we conduct an ablation study to isolate and quantify the performance contributions of the proposed techniques. This study evaluates the performance of the following ablation variants: the baseline FlashAttention-2 (FA2), the baseline FlashAttention-2 with the 16-row surrogate maximum $\hat{m}$ integrated (FA2+Max16), an all-tensorized variant of FlashAttention-T (AllTensor), and the complete ILP FlashAttention-T (FA-T). Note that this ablation study is conducted only on the Ampere GPUs, as the all-tensorized variant of FlashAttention-T is currently impossible to efficiently implement on the H100, due to the compiler-imposed limitation detailed in Section 5.

Numerical stability and accuracy. This category of evaluations examines the impacts of FlashAttention-T on numerical stability and accuracy. We first conduct a synthetic test on FlashAttention-T’s attention accuracy using random inputs Q, K, V sampled from distributions designed to simulate real-world attention input scenarios. We calculate the root-mean-square error (RMSE) between the attention output O and the FP64 reference output, then compare the RMSE values against ablation variants to assess accuracy and detect numerical failures (e.g. the all-underflow failure).

Additionally, to evaluate the real-world applicability of FlashAttention-T, we replace the PyTorch attention kernels with FlashAttention-T implementations and test them on full generative benchmarks using various LLMs. We report functional correctness metrics against the baselines and also detect any numerical failures.

6.3 Comparison Baselines

We compare FlashAttention-T against various baselines, consisting of both highly optimized libraries and a representative deep learning compiler. This selection aims to contextualize the performance against state-of-the-art practices and compiler-generated solutions. Specifically, our baselines include: (1) **FlashAttention-2 and FlashAttention-3**: These are the most prominent fused attention implementations and represent the strongest baselines on Ampere and Hopper GPUs, respectively [5, 26]. (2) **FlashInfer**: An emerging highly optimized library, designed specifically for efficient LLM inference [37]. (3) **Triton**: We use the reference Triton fused attention kernel [22] to assess FlashAttention-T’s performance against this deep learning compiler solution [29].

7 Experimental Results

7.1 Main Results

In this evaluation, we measure the attention throughput of FlashAttention-T and compare it against the baselines across various configurations. Following FlashAttention-3, we fix the total token count to 16384 and total hidden dimension to 2048, and vary the sequence length and head dimension². Results in Figure 8 show that FlashAttention-T outperforms all baselines in nearly all configurations. Against the strongest baselines FlashAttention-2 and FlashAttention-3, FlashAttention-T achieves speedups ranging from 1.02× to 1.19×, with mean speedups of 1.05–1.17× averaged over sequence lengths per configuration. Against FlashInfer and Triton, FlashAttention-T achieves average speedups of 1.03–1.39×. We observe the following across configurations: (1) **Causal masking**. FlashAttention-T mostly achieves higher speedups in non-causal configurations, as causal attention introduces masking overhead. (2) **Head dimension**. Speedups of FlashAttention-T are greater for smaller head dimensions, as larger head dimensions increase the GEMM FLOP count, except on AGX Orin, where the baselines are less efficient. (3) **Sequence length**. FlashAttention-T’s throughput gains remain mostly stable across different sequence lengths, as the repurposed tensor MMA instructions operate on register data and consistently reach peak throughput despite varying memory pressure impacting baseline GEMM throughputs.

7.2 Vector Interval Ratio Evaluation

To quantify how FlashAttention-T improves performance by reducing tensor unit idle time, we calculate the vector interval ratio $t'_{\text{vec}}/t'_{\text{iter}}$ of FlashAttention-T relative to the baseline across all evaluation platforms under various non-causal attention configurations. On the Ampere GPUs, where compiler optimizations in an ILP scheduling complicate direct measurement of t'_{vec} , we estimate the vector interval ratio using the formula $(t'_{\text{softmax}} - (t_{\text{vec}} - t'_{\text{softmax}}))/t'_{\text{iter}}$. Here, t'_{softmax} and t'_{iter} are FlashAttention-T’s measured softmax and iteration times, respectively, while t_{vec} is the baseline’s vector interval time. The saved softmax time $t_{\text{vec}} - t'_{\text{softmax}}$ reflects tensor unit utilization via repurposed tensor MMA instructions using ILP, thus, the remaining softmax time represents the vector interval time t'_{vec} . On the Hopper GPU, t'_{vec} is directly measured. Results in Figure 9 show that the ILP FlashAttention-T’s vector interval ratios are 1.17–2.18× lower than the baseline on Ampere GPUs, while the TLP FlashAttention-T shows an even stronger reduction down to 2.7% on the Hopper GPU. The TLP scheduling shows better efficiency, due to its more flexible dynamic tensor-vector overlapping unconstrained by static instruction ordering and reduced register pressure with the WGMMMA instructions.

²E.g., a sequence length s of 4096 and head dimension h of 64 yield a batch size of 4 and 32 heads.

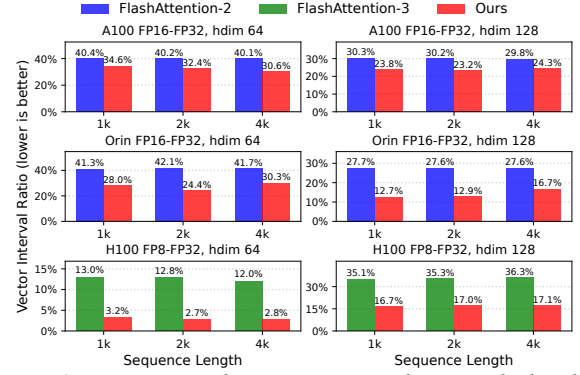


Figure 9. Vector interval ratio comparison between the baselines and FlashAttention-T. Note that the vector interval ratio results on Ampere GPU are estimated based on measured clock values.

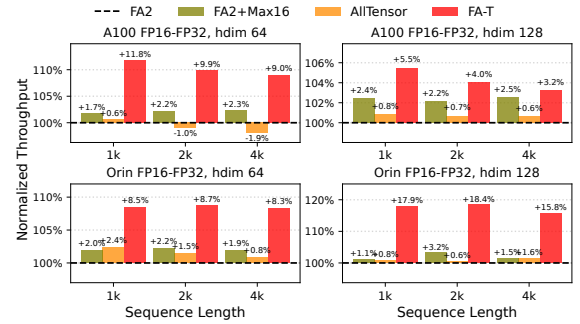


Figure 10. Attention throughput of FlashAttention-T and its ablation variants. The results are normalized to FlashAttention-2.

7.3 Ablation Study

To quantify the contributions of each proposed technique, we conduct an ablation study on the implementation variants using non-causal attention configurations shown in Figure 10. Results across the four experiment settings show similar trends: (1) FA2+Max16 achieves around 1% to 3% performance gains over FA2, as the surrogate $\hat{m}$ requires only two warp all-reduce REDUX instructions instead of multiple synchronous warp shuffle SHFL instructions. (2) AllTensor slightly degrades performance compared to FA2+Max16. This is because the repurposed tensor MMA instructions introduce swap overhead, while providing no increase in effective throughput compared to the vector instructions. (3) The ILP FA-T achieves a maximum performance gain of 18.4% over FlashAttention-2, demonstrating the effectiveness of our ILP scheduling technique.

7.4 Synthetic Attention Accuracy Evaluation

In this evaluation, we sample synthetic attention inputs Q , K , and V from the distribution $\mathcal{N}(0, 1) + \mathcal{N}(0, \tau^2) \cdot \text{Bernoulli}(\frac{1}{1000})$, and calculate output root-mean-square error (RMSE) between the ablation variants and the reference FP64 attention implementation, with $h=128$ and $s=4096$. This setting follows FlashAttention-3 [26], where the two terms represent the base distribution and an outlier distribution with variance τ^2 [28], respectively. Note the outliers are introduced sparsely via the Bernoulli component.

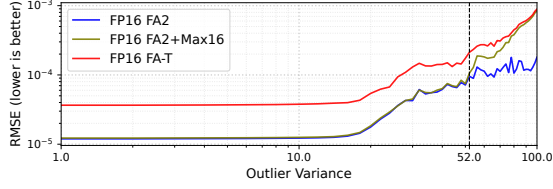


Figure 11. The RMSE for various ablation variants compared to the reference FP64 attention implementation on the A100 GPU.

Results in Figure 11 show that across all tested τ^2 values, the ILP FA-T achieves an RMSE less than 10^{-3} and is in the same order of magnitude as FA2, with no observed numerical failure. By comparing FA-T with FA2+Max16, we make two observations: (1) At $\tau^2 < 52$, FA-T’s RMSE is dominated by the numerical precision loss induced by the TF32 computation within the repurposed tensor MMA operations. (2) At $\tau^2 \geq 52$, the RMSE values of both FA2+Max16 and FA-T are dominated by the large numerical range of the attention scores incurred by large outlier values. Nevertheless, the results in the following evaluation confirm that our method does not affect real-world application accuracy. In addition, experimental results on H100 show that the TLP FlashAttention-T achieves RMSE values with a mean difference of 1.61×10^{-7} compared to the baseline FlashAttention-3 in FP8-FP32 attention configurations. This deviation is minimal since the surrogate maximum $\hat{m}$ is not used in our TLP implementation, and the TF32 attention score row-summation does not constitute a precision bottleneck for FP8-FP32 attention.

Table 2. Generative benchmark accuracy comparison

Benchmark	Model	FlashAttention-2	Ours	Ratio
HumanEval ^[4] Pass@10	Llama3.1 8B ^[8]	0.6889 ± 0.0286	0.6889 ± 0.0289	0.9999
	Ministral 8B ^[17]	0.8321 ± 0.0228	0.8383 ± 0.0217	1.0073
	Qwen3 8B ^[36]	0.8166 ± 0.0238	0.8177 ± 0.0239	1.0013
MMLU ^[10] Score	LLama2 13B ^[30]	0.5207 ± 0.0040	0.5212 ± 0.0040	1.0009
	Mistral NeMo ^[18]	0.6566 ± 0.0037	0.6566 ± 0.0037	1.0000
	Qwen3 14B ^[36]	0.7799 ± 0.0033	0.7804 ± 0.0032	1.0006

7.5 Generative Benchmark Accuracy Evaluation

To evaluate the real-world applicability of FlashAttention-T, we replace the PyTorch attention kernels with FlashAttention-T implementations and test them on two generative benchmarks using six LLMs listed in Table 2. Results show that FlashAttention-T achieves functional correctness metrics nearly identical to the baseline, with no numerical failures detected. This also suggests that large outlier attention values are rare in modern LLMs with the ubiquitous use [42] of normalization techniques such as RMSNorm [38].

8 Related Work

GPU tensor-vector parallelism. There is increasing research interest in exploiting tensor-vector parallelism on modern GPUs. Plasticine [41] leverages intra-SM parallelism

to utilize the tensor and vector units concurrently, but is limited to multi-task scenarios. FlashAttention-3 [26] leverages TLP for GEMM-softmax overlapping within attention computation, but leaves a vector interval bottleneck that FlashAttention-T addresses. Mosaic [35] exploits ILP between the tensor and vector units for efficient GEMM.

Attention variants. Attention mechanisms beyond Multi-Head Attention (MHA) [31] include Multi-Query Attention (MQA), Grouped Query Attention (GQA), and Multi-Latent Attention (MLA), which optimize attention structures for efficiency [2, 7, 27]. Instead, FlashAttention-T focuses on tensorizing softmax computation, making it orthogonal and applicable to all variants. Preliminary results indicate that FlashAttention-T achieves similar speedups on GQA as observed in our MHA evaluation, demonstrating its versatility.

Quantized attention. Recent works, such as SageAttention, advance low-precision attention by developing novel quantization techniques and utilizing FP8 GEMM in fused attention [39, 40]. In contrast, FlashAttention-T accelerates FP8-FP32 attention by tensorizing softmax computation, a complementary approach to the quantization efforts, aligning with the trend toward low-precision deep learning computation on modern GPUs.

9 Conclusion

This paper introduces FlashAttention-T, a novel fused attention implementation that advances toward fully tensorized attention. FlashAttention-T enables tensorizing critical softmax primitives within the fused attention algorithm with novel operand value assignment methods to repurpose GPU tensor MMA instructions and a tensorized online softmax algorithm. Building upon these, we propose architecture-aware scheduling techniques to parallelize the softmax computation across the tensor and vector units, exploiting tensor-vector parallelism for optimal performance. Experimental results show that FlashAttention-T outperforms the state-of-the-art FlashAttention-2 and FlashAttention-3 with average attention throughput speedups of 1.05–1.17× across diverse attention configurations on Ampere and Hopper GPUs, while preserving numerical stability and accuracy.

Data-Availability Statement

Artifact availability. The artifact for reproducing the main results in this paper is archived on Zenodo (version v1) [34].

Acknowledgements

We would like to extend our gratitude to our reviewers for their feedback and suggestions. This work is partially supported by the NSF of China (Grants No.U22A2028, 62302483), Strategic Priority Research Program of the Chinese Academy of Sciences (Grants No.XDB0660300, XDB0660302), CAS Project for Young Scientists in Basic Research (YSBR-029) and Youth Innovation Promotion Association CAS.

References

- [1] Hamdy Abdelkhalik, Yehia Arafat, Nandakishore Santhi, and Abdel-Hameed A. Badawy. 2022. Demystifying the Nvidia Ampere Architecture through Microbenchmarking and Instruction-level Analysis. In *IEEE High Performance Extreme Computing Conference, HPEC 2022, Waltham, MA, USA, September 19-23, 2022*. IEEE, 1–8. doi:10.1109/HPEC55821.2022.9926299
- [2] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, 4895–4901. doi:10.18653/V1/2023.EMNLP-MAIN.298
- [3] Shengnan An, Zexiong Ma, Zeqi Lin, Nanning Zheng, Jian-Guang Lou, and Weizhu Chen. 2024. Make Your LLM Fully Utilize the Context. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (Eds.). http://papers.nips.cc/paper_files/paper/2024/hash/71c3451f6cd6a4f82bb822db25cea4fd-Abstract-Conference.html
- [4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgens Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *CoRR* abs/2107.03374 (2021). arXiv:2107.03374 <https://arxiv.org/abs/2107.03374>
- [5] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. <https://openreview.net/forum?id=mZn2Xyh9Ec>
- [6] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh (Eds.). http://papers.nips.cc/paper_files/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html
- [7] DeepSeek-AI, Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fanyun Lin, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Hao Yang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jin Chen, Jingyang Yuan, Junjie Qiu, Junxiao Song, Kai Dong, Kaige Gao, Kang Guan, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruizhe Pan, Runxin Xu, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Size Zheng, T. Wang, Tian Pei, Tian Yuan, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Liu, Xin Xie, Xingkai Yu, Xinnan Song, Xinyi Zhou, Xinyu Yang, Xuan Lu, Xuecheng Su, Y. Wu, Y. K. Li, Y. X. Wei, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Zheng, Yichao Zhang, Yiliang Xiong, Yilong Zhao, Ying He, Ying Tang, Yishi Piao, Yixin Dong, Yixuan Tan, Yiyuan Liu, Yongji Wang, Yongqiang Guo, Yuchen Zhu, Yudian Wang, Yuheng Zou, Yukun Zha, Yunxian Ma, Yuting Yan, Yuxiang You, Yuxuan Liu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhewen Hao, Zhihong Shao, Zhiniu Wen, Zhipeng Xu, Zhongyu Zhang, Zhuoshu Li, Zihan Wang, Zihui Gu, Zilin Li, and Ziwei Xie. 2024. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. *CoRR* abs/2405.04434 (2024). arXiv:2405.04434 doi:10.48550/ARXIV.2405.04434
- [8] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, Danny Wyatt, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Francisco Guzmán, Frank Zhang, Gabriel Synaeva, Gabrielle Lee, Georgia Lewis Anderson, Govind Thattai, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel Kloumann, Ishan Misra, Ivan Evtimov, Jack Zhang, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jiemer van der Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Junteng Jia, Kalyan Vasuden Alwala, Karthik Prasad, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalla, Kushal Lakhotia, Lauren Rantala-Young, Laurens van der Maaten, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Landzaat, Luke de Oliveira, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Maria Tsimpoukelli, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melanie Kambadur, Mike Lewis, Min Si, Mitesh Kumar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bashlykov, Nikolay Bogoychev, Niladri Chatterji, Ning Zhang, Olivier Duchenne, Onur Çelebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajjwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira Cabral, Robert Stojnic, Roberta Raileanu, Rohan Maheswari, Rohit Girdhar, Rohit Patel, Romain Sauvestre, Ronnie Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Son, Kim, Sergey Edunov, Shaoqiang Nie, Sharan Narang, Sharath R-parthy, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla,

Stephane Collet, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkez, Vincent Gonguet, Virginie Do, Vish Voleti, Vitor Albiero, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenyan Fu, Whitney Meers, Xavier Martinet, Xiaodong Wang, Xiaofang Wang, Xiaoqing Ellen Tan, Xide Xia, Xinfeng Xie, Xuchao Jia, Xuewei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yi Wen, Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie Delpierre Coudert, Zheng Yan, Zhengxing Chen, Zoe Papakipos, Aaditya Singh, Aayushi Srivastava, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alexei Baevski, Allie Feinstein, Amanda Kallet, Amit Sangani, Amos Teo, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Dong, Annie Franco, Anuj Goyal, Aparajita Saraf, Arkabandhu Chowdhury, Ashley Gabriel, Ashwin Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardi, Bernie Huang, Beth Loyd, Beto De Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Ce Liu, Changhan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Cynthia Gao, Damon Civin, Dana Beaty, Daniel Kreymer, Daniel Li, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkan Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Eric-Tuan Le, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Filippos Kokkinos, Firat Ozgenel, Francesco Caggioni, Frank Kanayet, Frank Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Grant Herman, Grigory Sizonov, Guangyi Zhang, Guna Lakshminarayanan, Hakan Inan, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Hongyuan Zhan, Ibrahim Damlaj, Igor Molybog, Igor Tufanov, Ilias Leontiadis, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Janice Lam, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhee, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kai Wu, Kam Hou U, Karan Saxena, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, Kaushik Veeraraghavan, Kelly Michelen, Keqian Li, Kiran Jagadeesh, Kun Huang, Kunal Chawla, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madian Khabza, Manav Avalani, Manish Bhatt, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim Naumov, Maya Lathi, Meghan Keneally, Miao Liu, Michael L. Seltzer, Michal Valko, Michelle Restrepo, Mihir Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Jubert Hermoso, Mo Metanat, Mohammad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikhil Mehta, Nikolay Pavlovich Laptev, Ning Dong, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollar, Polina Zvyagina, Prashant Ratanchandani, Pritish Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Rangaprabhu Parthasarathy, Raymond Li, Rebekkah Hogan, Robin Battey, Rocky Wang, Russ Howes, Ruty Rinott, Sachin Mehta, Sachin Siby, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sar-gun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Mahajan, Saurabh

- Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy Zha, Shishir Patil, Shiva Shankar, Shuqiang Zhang, Shuqiang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta, Summer Deng, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Koehler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihailescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xiaocheng Tang, Xiaojian Wu, Xiaolan Wang, Xilun Wu, Xinbo Gao, Yaniv Kleinman, Yanjun Chen, Ye Hu, Ye Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu, Wang, Yu Zhao, Yuchen Hao, Yundi Qian, Yunlu Li, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, Zhiwei Zhao, and Zhiyu Ma. 2024. The Llama 3 Herd of Models. *CoRR* abs/2407.21783 (2024). arXiv:2407.21783 doi:10.48550/ARXIV.2407.21783
- [9] Zhong Guan, Likang Wu, Hongke Zhao, Ming He, and Jianping Fan. 2025. Attention Mechanisms Perspective: Exploring LLM Processing of Graph-Structured Data. *CoRR* abs/2505.02130 (2025). arXiv:2505.02130 doi:10.48550/ARXIV.2505.02130
- [10] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring Massive Multitask Language Understanding. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net. <https://openreview.net/forum?id=d7KBjml3GmQ>
- [11] Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, Kangdi Chen, Yuhang Dong, and Yu Wang. 2024. FlashDecoding++: Faster Large Language Model Inference with Asynchronization, Flat GEMM Optimization, and Heuristics. In *Proceedings of the Seventh Annual Conference on Machine Learning and Systems, MLSys 2024, Santa Clara, CA, USA, May 13-16, 2024*, Phillip B. Gibbons, Gennady Pekhimenko, and Christopher De Sa (Eds.). mlsys.org. https://proceedings.mlsys.org/paper_files/paper/2024/hash/5321b1dabdc2be188d796c21b733e8c7-Abstract-Conference.html
- [12] Jiri Hron, Yasaman Bahri, Jascha Sohl-Dickstein, and Roman Novak. 2020. Infinite attention: NNGP and NTK for deep attention networks. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, 4376–4386. <http://proceedings.mlr.press/v119/hron20a.html>
- [13] Aaron Jarmusch, Nathan Graddon, and Sunita Chandrasekaran. 2025. Dissecting the NVIDIA Blackwell Architecture with Microbenchmarks. *CoRR* abs/2507.10789 (2025). arXiv:2507.10789 doi:10.48550/ARXIV.2507.10789
- [14] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, Jason Flinn, Margo I. Seltzer, Peter Druschel, Antoine Kaufmann, and Jonathan Mace (Eds.). ACM, 611–626. doi:10.1145/3600006.3613165
- [15] Weile Luo, Ruibo Fan, Zeyu Li, Dayou Du, Qiang Wang, and Xiaowen Chu. 2024. Benchmarking and Dissecting the Nvidia Hopper GPU Architecture. In *IEEE International Parallel and Distributed Processing Symposium, IPDPS 2024, San Francisco, CA, USA, May 27-31, 2024*. IEEE, 656–667. doi:10.1109/IPDPS57955.2024.00064
- [16] Maxim Milakov and Natalia Gimelshein. 2018. Online normalizer calculation for softmax. *CoRR* abs/1805.02867 (2018). arXiv:1805.02867 <http://arxiv.org/abs/1805.02867>

- [17] Mistral AI. 2024. Un Ministral, des Ministraux. Online. <https://mistral.ai/news/ministral> Model card available at: <https://huggingface.co/mistralai/Ministral-8B-Instruct-2410>.
- [18] Mistral AI and NVIDIA. 2024. Mistral NeMo 12B: A State-of-the-Art Language Model. Online. <https://mistral.ai/news/mistral-nemo> Model card available at: <https://huggingface.co/mistralai/Mistral-Nemo-Instruct-2407>.
- [19] NVIDIA Corporation. 2020. *NVIDIA A100 Tensor Core GPU Architecture*. Technical Report. NVIDIA. <https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf> Accessed: 2025-08-06.
- [20] NVIDIA Corporation. 2022. *NVIDIA H100 Tensor Core GPU Architecture*. Technical Report. NVIDIA. <https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c> Accessed: 2025-08-06.
- [21] NVIDIA Corporation. 2025. *Parallel Thread Execution (PTX) ISA Version 9.0*. <https://docs.nvidia.com/cuda/parallel-thread-execution/index.html> Accessed: 2025-08-25.
- [22] OpenAI. 2025. Fused Attention Tutorial. <https://triton-lang.org/main/getting-started/tutorials/06-fused-attention.html>. Accessed: 2025-08-11.
- [23] OpenAI. 2025. gpt-oss-120 & gpt-oss-20b Model Card. https://cdn.openai.com/pdf/419b6906-9da6-406c-a19d-1bb078ac7637/oai_gpt-oss_model_card.pdf. Accessed: 2025-08-11.
- [24] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Z. Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d'Alché-Buc, Emily B. Fox, and Roman Garnett (Eds.). 8024–8035. <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee792f2bfa9f7012727740-Abstract.html>
- [25] Richard GE Pinch. 2005. The distance of a permutation from a subgroup of S_n . *arXiv preprint math/0511501* (2005).
- [26] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (Eds.). http://papers.nips.cc/paper_files/paper/2024/hash/7ede97c3e082c6df10a8d6103a2eebd2-Abstract-Conference.html
- [27] Noam Shazeer. 2019. Fast Transformer Decoding: One Write-Head is All You Need. *CoRR* abs/1911.02150 (2019). arXiv:1911.02150 <http://arxiv.org/abs/1911.02150>
- [28] Mingjie Sun, Xinlei Chen, J. Zico Kolter, and Zhuang Liu. 2024. Massive Activations in Large Language Models. *CoRR* abs/2402.17762 (2024). arXiv:2402.17762 doi:10.48550/ARXIV.2402.17762
- [29] Philippe Tillet, Hsiang-Tsung Kung, and David D. Cox. 2019. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages, MAPL@PLDI 2019, Phoenix, AZ, USA, June 22, 2019*, Tim Mattson, Abdullah Muzahid, and Armando Solar-Lezama (Eds.). ACM, 10–19. doi:10.1145/3315508.3329973
- [30] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shrutu Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton-Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xi-ang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurélien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *CoRR* abs/2307.09288 (2023). arXiv:2307.09288 doi:10.48550/ARXIV.2307.09288
- [31] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (Eds.). 5998–6008. <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- [32] Suyuchen Wang, Ivan Kobayev, Peng Lu, Mehdi Rezagholizadeh, and Bang Liu. 2024. Resonance RoPE: Improving Context Length Generalization of Large Language Models. In *Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, 586–598. doi:10.18653/V1/2024.FINDINGS-ACL.32
- [33] Xindi Wang, Mahsa Salmani, Parsa Omid, Xiangyu Ren, Mehdi Rezagholizadeh, and Armaghan Eshaghi. 2024. Beyond the Limits: A Survey of Techniques to Extend the Context Length in Large Language Models. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI 2024, Jeju, South Korea, August 3-9, 2024*. ijcai.org, 8299–8307. <https://www.ijcai.org/proceedings/2024/917>
- [34] Jianxing Xu, Yuanbo Wen, Jun Bi, and Ruibai Xu. 2025. Artifact for the PPOPP '26 paper: "FlashAttention-T: Towards Fully Tensorized Attention by Exploiting Tensor-Vector Parallelism." Zenodo. doi:10.5281/zenodo.17673796 Version v1, published 2025-11-22.
- [35] Jianxing Xu, Yuanbo Wen, Zikang Liu, Ruibai Xu, Tingfeng Ruan, Jun Bi, Rui Zhang, Di Huang, Xinkai Song, Yifan Hao, Xing Hu, Zidong Du, Chongqing Zhao, Jiang Jie, and Qi Guo. 2025. Mosaic: Exploiting Instruction-Level Parallelism on Deep Learning Accelerators with iTex Tessellation. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2025, Rotterdam, Netherlands, 30 March 2025 - 3 April 2025*, Lieven Eeckhout, Georgios Smaragdakis, Katai Liang, Adrian Sampson, Martha A. Kim, and Christopher J. Rossbach (Eds.). ACM, 672–688. doi:10.1145/3676641.3716262
- [36] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jian Yang, Jiayi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. *CoRR* abs/2505.09388 (2025). arXiv:2505.09388 doi:10.48550/ARXIV.2505.09388

- [37] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. *CoRR* abs/2501.01005 (2025). arXiv:2501.01005 doi:10.48550/ARXIV.2501.01005
- [38] Biao Zhang and Rico Sennrich. 2019. Root Mean Square Layer Normalization. In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d'Alché-Buc, Emily B. Fox, and Roman Garnett (Eds.). 12360–12371. <https://proceedings.neurips.cc/paper/2019/hash/1e8a19426224ca89e83cef47f1e7f53b-Abstract.html>
- [39] Jintao Zhang, Haofeng Huang, Pingle Zhang, Jia Wei, Jun Zhu, and Jianfei Chen. 2025. Sageattention2: Efficient attention with thorough outlier smoothing and per-thread int4 quantization. In *International Conference on Machine Learning (ICML)*.
- [40] Jintao Zhang, Jia wei, Pingle Zhang, Jun Zhu, and Jianfei Chen. 2025. SageAttention: Accurate 8-Bit Attention for Plug-and-play Inference Acceleration. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net. <https://openreview.net/forum?id=OL44KtasKc>
- [41] Han Zhao, Weihao Cui, Quan Chen, Jieru Zhao, Jingwen Leng, and Minyi Guo. 2021. Exploiting Intra-SM Parallelism in GPUs via Persistent and Elastic Blocks. In *39th IEEE International Conference on Computer Design, ICCD 2021, Storrs, CT, USA, October 24-27, 2021*. IEEE, 290–298. doi:10.1109/ICCD53106.2021.00054
- [42] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. 2023. A Survey of Large Language Models. *CoRR* abs/2303.18223 (2023). arXiv:2303.18223 doi:10.48550/ARXIV.2303.18223
- [43] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark W. Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (Eds.). http://papers.nips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html

Received 2025-09-01; accepted 2025-11-10